

MongoDB

Fiche de preparation pour l'expose

Comprendre • utiliser • modeliser • optimiser • distribuer •
mesurer

Objectif : etre capable d'expliquer MongoDB sans reciter les diapositives, et de repondre aux questions techniques classiques.

1. Le fil conducteur de l'expose

Le plus important est de comprendre la logique globale plutot que de memoriser chaque diapositive.

- MongoDB → base NoSQL orientee documents
- Database → Collection → Document BSON → Field
- CRUD → Requetes → Index → Aggregation
- Modelisation → Transactions → Replica Set → Sharding
- MongoDB + Redis → Benchmark → Interpretation des resultats

Phrase-res ume

MongoDB est une base NoSQL orientee documents qui permet de stocker et interroger des donnees flexibles, avec des mecanismes d'indexation, d'agregation, de replication et de distribution pour construire des applications allant d'un petit projet a des architectures distribuees.

2. Qu'est-ce que MongoDB ?

- Systeme de gestion de base de donnees NoSQL oriente documents.
- Donnees stockees sous forme de documents BSON, proches du JSON.
- Documents regroupes dans des collections.
- Possibilite de représenter des structures imbriquees et des tableaux.
- Utilisable en local ou dans le cloud avec MongoDB Atlas.

Vocabulaire a connaitre

Terme	Signification
Database	Base de donnees
Collection	Ensemble de documents
Document	Enregistrement BSON
Field	Champ / propriete
BSON	Format binaire utilise par MongoDB
_id	Identifiant du document

3. Comprendre les documents et le CRUD

Exemple d'un utilisateur :

```
{
  "_id": "user:42",
  "name": "Alice",
  "age": 25,
  "address": { "city": "Paris", "zip": "75001" },
  "skills": ["Python", "MongoDB"]
}
```

CRUD = les 4 operations de base

```
// CREATE
db.users.insertOne({ name: "Alice", age: 25 })

// READ
db.users.find({ age: { $gte: 18 } })

// UPDATE
db.users.updateOne(
  { name: "Alice" },
  { $set: { age: 26 } }
)

// DELETE
db.users.deleteOne({ name: "Alice" })
```

A l'oral

Explique toujours “ce que fait la commande” puis “pourquoi on l'utilise”.

4. Fil rouge : la boutique en ligne

Garde cet exemple en tete pendant tout l'expose.

- users : profil, email, adresse, preferences.
- products : nom, prix, categorie, stock.
- orders : client, lignes de commande, montant, statut.

```
{
  "_id": "order:9001",
  "userId": "user:42",
  "products": [
    { "productId": "product:123", "quantity": 1, "price": 1299.90 }
  ],
  "total": 1299.90,
  "status": "paid"
}
```

- 1. L'utilisateur consulte un produit.
- 2. Il passe une commande.
- 3. Le stock est decremente.
- 4. La commande devient "paid".
- 5. On analyse les ventes avec aggregation.

5. Modeliser et optimiser les requetes

Embedded document ou Reference ?

- Embedded : placer les donnees liees dans le meme document. Utile lorsqu'elles sont naturellement lues ensemble.
- Reference : stocker un identifiant vers un autre document. Utile lorsque les donnees sont partagees, independantes ou volumineuses.

Question a se poser "Comment l'application va-t-elle lire et modifier ces donnees ?"

Index

```
db.users.createIndex({ email: 1 })
```

- Un index peut rendre certaines requetes plus efficaces.
- Un index consomme de l'espace et ajoute un cout aux ecritures / mises a jour.
- Utilise explain() pour comprendre le plan d'execution.

Aggregation

```
db.orders.aggregate([
  { $match: { status: "paid" } },
  { $group: { _id: "$userId", total: { $sum: "$amount" } } },
  { $sort: { total: -1 } }
])
```

Lecture mentale : filtrer → regrouper / calculer → trier.

6. Architecture et mise a l'échelle

Architecture applicative

Utilisateur → Application / API → Driver MongoDB → MongoDB

Replica Set

- PRIMARY : recoit les ecritures selon la configuration.
- SECONDARY : replique les donnees.
- Objectif principal : haute disponibilite et resilience, pas simplement "plus de vitesse".

Sharding

- Distribue les donnees sur plusieurs shards.
- La shard key influence la distribution et le routage des requetes.
- Interet : faire evoluer l'architecture horizontalement lorsque l'échelle le justifie.
- Compromis : distribution + complexite d'architecture.

Transactions

- Une transaction permet de considerer plusieurs modifications comme une meme unite logique, par exemple creer une commande et diminuer le stock.

7. MongoDB + Redis

Ne presente pas Redis comme “MongoDB mais plus rapide”. Les deux peuvent avoir des roles differents.

Sujet	MongoDB	Redis
Modele	Documents BSON	Cle → valeur / structures
Usage frequent	Donnees metier persistantes	Cache, sessions, compteurs, temps reel
Acces	Requetes + aggregation	Commandes orientees cles / structures
Architecture	Replica sets + sharding	Replication + Redis Cluster

Architecture complete Application → Redis (cache) → MongoDB (donnees metier). Redis peut eviter de relire MongoDB pour des donnees tres demandees.

Benchmark

- Workload identique : write, read, update, delete.
- Mesures : ops/s (debit), p50 (latence mediane), p95 (latence haute).
- Conditions controlees : meme machine, memes donnees, meme protocole, memes tailles de payload, memes reglages pertinents.
- Les chiffres doivent etre presentes comme des resultats d'un scenario precis, pas comme un classement universel.

8. Questions probables du professeur

“MongoDB, c’est quoi ?”

Une base NoSQL orientée documents qui stocke ses données sous forme de documents BSON regroupés dans des collections.

“Pourquoi utiliser des documents ?”

Pour représenter naturellement des structures riches, imbriquées et proches des objets manipulés par l’application.

“Quelle différence entre collection et document ?”

Une collection regroupe des documents ; un document correspond à un enregistrement.

“À quoi sert un index ?”

À rendre certaines requêtes plus efficaces en fournissant une structure de recherche dédiée.

“Faut-il mettre des index partout ?”

Non. Ils consomment de l’espace et ont un coût sur les écritures ; ils doivent correspondre aux requêtes réelles.

“MongoDB supporte les transactions ?”

Oui, y compris des transactions multi-documents.

“Replica Set = performance ?”

Principalement haute disponibilité et réplication.

“Sharding = performance automatique ?”

Non. Il distribue les données, mais ajoute aussi de la complexité ; l’impact dépend de l’architecture et du workload.

“MongoDB ou Redis ?”

Souvent des rôles différents : MongoDB pour les données documentaires métier ; Redis pour des accès mémoire rapides, cache, sessions, compteurs, etc.

“Quelle base est la plus rapide ?”

Il n’existe pas de réponse universelle. Il faut définir un workload et mesurer dans des conditions contrôlées.

9. Pièges à éviter et notions à connaître

Evite ces affirmations trop simplistes :

- “MongoDB est une base SQL plus rapide” → faux / reducteur.
- “NoSQL signifie sans structure” → faux.
- “MongoDB n’a pas de transactions” → faux.
- “Redis remplace MongoDB” → pas nécessairement.
- “Le sharding rend automatiquement MongoDB plus rapide” → faux.
- “Plus d’index = toujours plus de performance” → faux.

A connaître par cœur

- NoSQL
- Document
- Collection
- BSON
- CRUD
- Query
- Index
- Aggregation
- Embedded document
- Reference
- Replica Set
- Sharding
- Redis
- Benchmark
- p50
- p95
- ops/s

Phrase finale pour l’oral

MongoDB est avant tout un modèle de données et une architecture : sa valeur dépend de la façon dont on l’utilise.

10. Sources officielles

- MongoDB Manual — <https://www.mongodb.com/docs/manual/>
- MongoDB Replication — <https://www.mongodb.com/docs/manual/replication/>
- MongoDB Sharding — <https://www.mongodb.com/docs/manual/sharding/>
- MongoDB Atlas — <https://www.mongodb.com/docs/atlas/>
- Redis Data Types — <https://redis.io/docs/latest/develop/data-types/>
- Redis Persistence — https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/
- Redis Benchmarks — https://redis.io/docs/latest/operate/oss_and_stack/management/optimization/benchmarks/

Note : les caracteristiques et recommandations techniques peuvent evoluer selon les versions ; verifier la documentation correspondant a la version utilisee pour l'expose.